Transit



FRONT HTML · CSS · JS | BACK PHP 8.2 + PDO | BASE MariaDB 10.4 | SERVEUR XAMPP

Le front saisit, le back décide, la base garantit. Seul PHP connaît les identifiants SQL.

ARCHITECTURE BDD · DÉMARCHE ET MODÈLE CHEN

Où stocker la quantité mise en colis ?



COLIS — **0,N ◇ 0,N** — **PRÉLÈVEMENT**

Un prélèvement de **12 unités** est réparti entre deux colis. Le premier en reçoit 7, le second 5.

La quantité appartient au couple (colis, prélèvement) : la relation N:N devient la table **package_pick**.

Démarche : cas d’utilisation UML (5 acteurs) → MCD Chen → schéma relationnel (43 tables, 68 clés étrangères) → SQL et données factices.

ARCHITECTURE BDD · CHOIX DE CONCEPTION

Quatre choix qui structurent la base.

Stock = historique

Le stock est la somme des mouvements, jamais modifiés : on sait toujours pourquoi il vaut ce qu'il vaut.

Lot ≠ palette

Chaque palette reçue garde sa réception et son camion : la provenance d'un lot n'est jamais perdue.

N:N → table

package_pick porte la quantité répartie entre un colis et un prélèvement (12 = 7 + 5).

Rien ne s'efface

Aucun ON DELETE CASCADE, journal non modifiable. 3NF, avec 4 dénormalisations justifiées.

43 tables, 68 clés étrangères, 22 contraintes CHECK. PHP sans framework : chaque requête SQL est écrite et visible.

ARCHITECTURE BDD · NORMALISATION

Une information à un seul endroit.

1NF

Valeurs atomiques : les équipements d'un quai sont des lignes de **dock_equipment**, pas une liste « niveleur, hayon » dans une colonne.

2NF

Clé (avis, article) → quantité attendue. Le nom de l'article ne dépend que de l'article : il reste dans **article**.

3NF

Pas de dépendance transitive : la zone d'un emplacement se retrouve par jointure (niveau → rack → allée → zone), jamais recopiée.

Quatre dénormalisations assumées et contrôlées : le solde de stock, l'état courant d'un passage, son chauffeur et son véhicule, l'attendu par livraison.

ARCHITECTURE BDD · AUTOMATISATION

Qui déclenche quoi ?

Trois façons de faire travailler la base, vues dans le TD Automatisation.

À chaque écriture

127 triggers : contrôler, journaliser, calculer le stock. Personne ne les appelle.

Quand on l'appelle

3 procédures et 1 fonction, appelées par le site avec CALL ou SELECT.

À heure fixe

1 évènement : chaque nuit à 2 h, sans aucun utilisateur connecté.

Toute écriture du site passe par une **transaction** : COMMIT si tout réussit, ROLLBACK à la moindre erreur.

ARCHITECTURE BDD · TRIGGERS

La base se protège seule.

BEFORE INSERT / UPDATE

Avant l'écriture, il peut refuser : **SIGNAL** bloque un rendez-vous qui chevauche un autre sur le même quai.

AFTER INSERT / UPDATE / DELETE

Après l'écriture, il réagit : OLD et NEW vont dans **audit_log**.

AFTER INSERT

Chaque mouvement met à jour **stock_balance**.

BEFORE UPDATE / DELETE

Sur le journal : refusés, l'historique est immuable.

Sur le site : à chaque enregistrement, sans une ligne de PHP. 127 triggers : 3 de cohérence, 11 de protection, 4 de calcul, 109 d'audit.

PHPMYADMIN TABLE APPOINTMENT · DÉCLENCHEURS

Nom	Temps	Évènement
appointment_no_overlap_insert	BEFORE	INSERT
appointment_no_overlap_update	BEFORE	UPDATE
audit_appointment_delete	AFTER	DELETE
audit_appointment_insert	AFTER	INSERT
audit_appointment_update	AFTER	UPDATE

ARCHITECTURE BDD · PROCÉDURES ET FONCTION SQL

Du SQL réutilisable, rangé dans la base.

`lock_site()`

Verrou SELECT … FOR UPDATE.

Site : début de chaque écriture.

`trace_lot(lot)`

Parcours complet d'un lot.

Site : page Traçabilité.

`stock_disponible()`

Fonction RETURNS INT.

Site : préparation de commande.

`archive_appointments()`

Archive dans une transaction.

Appelée par l'évènement.

Une **procédure** exécute un traitement (CALL) ; une **fonction** renvoie une valeur utilisable dans un SELECT.

PHPMYADMIN PROCÉDURES STOCKÉES

Nom	Type	Retourne
archive_appointments	PROCEDURE	
lock_site	PROCEDURE	
stock_disponible	FUNCTION	int(11)
trace_lot	PROCEDURE	

ARCHITECTURE BDD · ÉVÈNEMENT

Chaque nuit, sans personne.

Quand

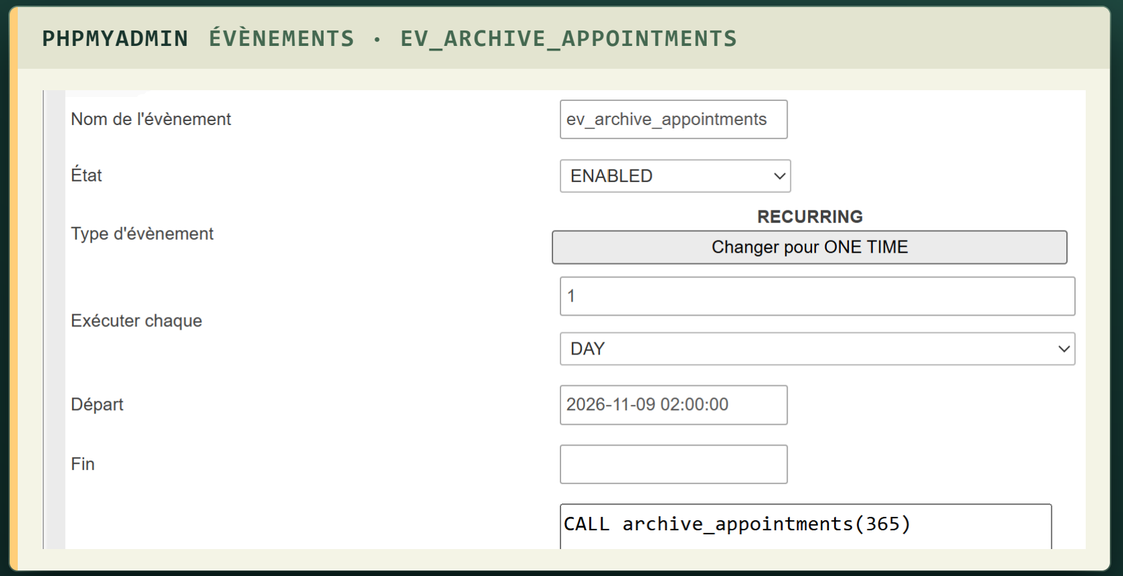
EVERY 1 DAY, à 2 h. Il faut event_scheduler=ON : notre script de démarrage le fait.

Quoi

CALL archive_appointments(365) : rendez-vous de plus d'un an sans passage de camion, dans une transaction.

Sur le site

Rien à cliquer. La vue **v_appointment_all** réunit courants et archivés (UNION ALL).



ARCHITECTURE BDD · VUES ET CALCULS

Calculer plutôt que stocker.

v_visit

Attente et immobilisation de chaque camion.

Site : Indicateurs.

Retard

Arrivée comparée au rendez-vous, par PHP.

Site : Passages.

v_receipt_discrepancy

Manquants, excédents, avariés.

Site : réception.

v_stock

Physique, réservé, disponible.

Site : Stock, FEFO.

Pas de trigger pour ces calculs : ils se déduisent des données présentes ; les stocker créerait une redondance. 11 vues, 9 utilisées par le site.

PHPMYADMIN LES 11 VUES



BACK-END · FONCTIONS PHP

Une classe PHP par métier.

Database

run(), all(), one() : requêtes préparées PDO. transaction() : BEGIN, CALL lock_site, COMMIT ou ROLLBACK.

Auth

register() : compte en attente, password_hash. login() : password_verify. can() : droits revérifiés à chaque page.

Warehouse

appointment(), arrive(), validateReceipt(), suggestions(), transfer(), prepare() en FEFO, ship().

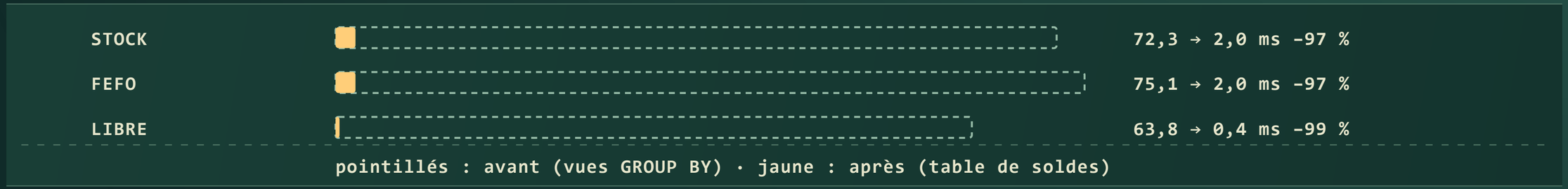
Catalog · Reports

save() et delete() : ajout, modification, suppression contrôlés. Recherches multicritères, export CSV.

Contrôle des saisies côté serveur (dates, quantités, compatibilités) ; jeton CSRF sur chaque formulaire ; h() échappe chaque affichage.

ARCHITECTURE BDD · OPTIMISATION

Combien coûte une recherche de stock ?



Pourquoi

EXPLAIN : chaque recherche recalculait le solde par GROUP BY sur les 24 917 mouvements (table dérivée lue en entier).

Comment

Une table de soldes tenue à jour par trigger, comme les statistiques du TD ; un index composé pour les créneaux (7,2 → 1,4 ms).

Contrepartie

Une redondance contrôlée : pas de surcoût mesurable en écriture, solde comparé au recalcul depuis l'historique.

Sur le site : pages Stock, préparation et transferts. Démarche du TD : mesurer, lire EXPLAIN, optimiser, mesurer à nouveau (médianes de 15 exécutions).